DSA Track

Interview Preparation Series • Professional Edition

DSA Patterns & Templates

01

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

> The "when you see X → use Y" trigger guide for FAANG coding rounds. Dense, copy-pasteable, built for fast recognition and last-minute revision.

1 Overview

1.1 How to Use This File

- **Before interview**: read the Pattern Recognition Cheat Table (§2) until you can map a problem sentence to a pattern in under 10 seconds.
- **During prep**: for each pattern in §3, memorize the trigger signal, copy the template, and solve the listed problems.
- **Night before**: skim the Revision Cheat Sheet (§7).

1.2 Universal Problem-Solving Workflow

```python
1. CLARIFY    — constraints (N size), edge cases, return type, mutate or not?
2. EXAMPLES   — run 2-3 examples by hand, including edge cases
3. BRUTE FORCE— state it out loud, give its complexity
4. OPTIMIZE   — identify the bottleneck; apply pattern recognition
5. CODE       — write clean code with variable names that tell the story
6. TEST       — trace through your examples + edge cases on the code
7. COMPLEXITY — state time and space, justify each term
```

> **Rule of thumb**: always state brute force before optimizing. Interviewers reward structured thinking over jumping to solutions.

2 Pattern Recognition Cheat Table

> The most important section. Map the problem's key phrases to a pattern before writing a single line of code.

If the problem says / asks for ...	Reach for this pattern
Sorted array, find pair/triplet with target sum	**Two Pointers**
Remove duplicates in-place, sorted array	**Two Pointers**
Longest/shortest subarray/substring with condition	**Sliding Window (variable)**
Subarray of exact size K with property	**Sliding Window (fixed)**
Linked list cycle, middle of list, palindrome LL	**Fast & Slow Pointers**
Overlapping intervals, merge/insert intervals	**Merge Intervals**
Array 0..N-1, find missing/duplicate number	**Cyclic Sort**
Reverse a linked list, K-group reversal	**In-place LL Reversal**
Shortest path, level-order traversal, minimum steps	**BFS**
All paths, connected components, flood fill	**DFS / Backtracking**
All subsets, all permutations, all combinations	**Backtracking / Subsets**

If the problem says / asks for ...	Reach for this pattern
"Find minimum/maximum satisfying condition", search space is monotonic	**Binary Search on Answer**
Top K largest/smallest, K closest, Kth element	**Heap (Top-K)**
Merge K sorted lists/arrays	**K-way Merge (Heap)**
Median of stream, two halves balanced	**Two Heaps**
Next greater element, stock prices, daily temperatures	**Monotonic Stack**
Subarray sum equals K, running sum queries	**Prefix Sum**
Connected components, union/cycle in graph	**Union-Find (DSU)**
Course schedule, dependency order, DAG ordering	**Topological Sort**
Word search, prefix matching, autocomplete	**Trie**
Count ways, min cost, max value, overlapping sub-problems	**Dynamic Programming**
0/1 choice per item, weight/capacity constraint	**DP — 0/1 Knapsack**
Unlimited copies of items allowed	**DP — Unbounded Knapsack**
Longest common subsequence/substring	**DP — LCS**
Longest increasing subsequence	**DP — LIS**
Grid, min path, unique paths	**DP — Matrix/Grid**
Intervals, burst balloons, matrix chain	**DP — Interval DP**
Locally optimal choice leads to global optimum	**Greedy**
XOR, AND, OR, bit shift tricks, unique element	**Bit Manipulation**

3 The Patterns

3.1 Two Pointers

Trigger / When to Use

> **When you see**: sorted array/string, find a pair or triplet, remove duplicates in-place, partition array.

Core Idea Use two indices that move toward each other (or in the same direction) to eliminate the need for a nested loop. Works on sorted input or with a logical invariant.

Template

Python

```python
def two_pointers(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return [left, right]          # found
        elif current < target:
            left += 1                     # need larger sum
        else:
            right -= 1                    # need smaller sum
    return []

# Same-direction variant (remove duplicates)
def remove_duplicates(arr):
```

```
    slow = 0
    for fast in range(1, len(arr)):
        if arr[fast] != arr[slow]:
            slow += 1
            arr[slow] = arr[fast]
    return slow + 1
```

Complexity: Time O(N), Space O(1)

Classic Problems

- Two Sum II (sorted array) — LeetCode 167
- 3Sum — LeetCode 15
- Container With Most Water — LeetCode 11
- Remove Duplicates from Sorted Array — LeetCode 26

Pitfalls

- Forgetting to skip duplicate values in 3Sum (leads to duplicate triplets).
- Using on unsorted input without sorting first (add O(N log N) sort cost).
- Off-by-one: `left < right` vs `left ⩽ right`.

3.2 Sliding Window (Fixed + Variable)

Trigger / When to Use

> **When you see**: contiguous subarray/substring, fixed window of size K, longest/shortest window satisfying a condition.

Core Idea Maintain a window `[left, right]` over the sequence. Expand right to grow; shrink left when the constraint is violated. Avoids recomputing the full window from scratch each step.

Template — Fixed Window (size K)

Python

```python
def fixed_window(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]   # slide: add right, remove left
        max_sum = max(max_sum, window_sum)
    return max_sum
```

Template — Variable Window

Python

```python
from collections import defaultdict

def variable_window(s, k):
    """Longest substring with at most k distinct characters."""
    char_count = defaultdict(int)
    left = 0
    max_len = 0
    for right in range(len(s)):
        char_count[s[right]] += 1
```

```
        while len(char_count) > k:          # shrink until valid
            char_count[s[left]] -= 1
            if char_count[s[left]] == 0:
                del char_count[s[left]]
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

Complexity: Time O(N), Space O(K) for the hash map

Classic Problems

- Maximum Sum Subarray of Size K
- Longest Substring Without Repeating Characters — LeetCode 3
- Minimum Window Substring — LeetCode 76
- Fruit Into Baskets — LeetCode 904

Pitfalls

- Shrinking the window with `if` instead of `while` (can leave window invalid).
- Not updating the result both inside and after the while shrink loop.
- Variable window: forgetting to delete keys with count 0 (artificially inflates distinct count).

3.3 Fast & Slow Pointers (Cycle Detection)

Trigger / When to Use

> **When you see**: linked list cycle, find middle node, palindrome linked list, detect duplicate in array (Floyd's algorithm).

Core Idea Two pointers move at different speeds (fast = 2 steps, slow = 1 step). If there's a cycle they must meet. Without a cycle, fast reaches the end first.

Template

Python

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

def find_cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            break
    else:
```

```
        return None          # no cycle
    # Reset one pointer to head; advance both at speed 1
    slow = head
    while slow is not fast:
        slow = slow.next
        fast = fast.next
    return slow              # cycle start

def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow              # middle node (for even length: second middle)
```

Complexity: Time O(N), Space O(1)

Classic Problems

- Linked List Cycle — LeetCode 141
- Linked List Cycle II (find start) — LeetCode 142
- Middle of the Linked List — LeetCode 876
- Find the Duplicate Number — LeetCode 287

Pitfalls

- Checking `slow == fast` before moving on first iteration (they start equal).
- Not handling `head is None` or single-node lists.
- Find Duplicate: the array must be treated as a linked list (index → value → next index).

3.4 Merge Intervals

Trigger / When to Use

> **When you see**: overlapping intervals, schedule conflicts, insert interval, minimum meeting rooms.

Core Idea Sort intervals by start time. Walk through; if the current interval overlaps the last merged one, merge by extending the end. Otherwise, append as new.

Template

Python

```
def merge_intervals(intervals):
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start ≤ merged[-1][1]:          # overlap: start ≤ prev end
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

def insert_interval(intervals, new_interval):
    result = []
    i = 0
```

```
    n = len(intervals)
    # Add all intervals that end before new_interval starts
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i]); i += 1
    # Merge all overlapping
    while i < n and intervals[i][0] ⩽ new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1
    result.append(new_interval)
    result.extend(intervals[i:])
    return result
```

Complexity: Time O(N log N) (sort), Space O(N)

Classic Problems

- Merge Intervals — LeetCode 56
- Insert Interval — LeetCode 57
- Meeting Rooms II (min rooms) — LeetCode 253
- Non-overlapping Intervals — LeetCode 435

Pitfalls

- Forgetting to sort before merging.
- Using < instead of ⩽ for overlap check (intervals that just touch should merge if problem says so).
- Mutating the original list element; use `merged[-1][1] = max(...)`.

3.5 Cyclic Sort

Trigger / When to Use

> **When you see**: array containing numbers in range `[1, N]` or `[0, N-1]`, find missing/duplicate/all missing numbers.

Core Idea Each number belongs at index `num - 1`. Walk the array; if `arr[i]` is not at its correct index, swap it there. After one pass, scan for misplaced numbers.

Template

Python

```python
def cyclic_sort(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1              # where nums[i] should live
        if nums[i] ≠ nums[correct]:        # not in place
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    return nums

def find_missing(nums):
    cyclic_sort(nums)
    for i, num in enumerate(nums):
        if num ≠ i + 1:
            return i + 1
    return len(nums) + 1
```

```
def find_duplicate(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != i + 1:
            if nums[i] == nums[correct]:    # duplicate found
                return nums[i]
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    return -1
```

Complexity: Time O(N), Space O(1)

Classic Problems

- Missing Number — LeetCode 268
- Find All Numbers Disappeared in an Array — LeetCode 448
- Find the Duplicate Number — LeetCode 287
- First Missing Positive — LeetCode 41

Pitfalls

- Infinite loop if you don't guard against `nums[i] == nums[correct]` when finding duplicates.
- Range `[0, N-1]` vs `[1, N]` — adjust `correct` index accordingly.

3.6 In-place Linked List Reversal

Trigger / When to Use

> **When you see**: reverse a linked list, reverse in groups of K, reverse between positions L and R.

Core Idea Iteratively redirect `next` pointers using three pointers: `prev`, `curr`, `next_node`. No extra space needed.

Template

Python

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    return prev

def reverse_sublist(head, left, right):
    """Reverse nodes from position left to right (1-indexed)."""
    dummy = ListNode(0)
    dummy.next = head
    prev = dummy
    for _ in range(left - 1):
        prev = prev.next
    curr = prev.next
```

```
    for _ in range(right - left):
        next_node = curr.next
        curr.next = next_node.next
        next_node.next = prev.next
        prev.next = next_node
    return dummy.next

def reverse_k_groups(head, k):
    """Reverse every k nodes."""
    dummy = ListNode(0)
    dummy.next = head
    group_prev = dummy
    while True:
        kth = get_kth(group_prev, k)
        if not kth:
            break
        group_next = kth.next
        prev, curr = kth.next, group_prev.next
        while curr ≠ group_next:
            tmp = curr.next
            curr.next = prev
            prev = curr
            curr = tmp
        tmp = group_prev.next
        group_prev.next = kth
        group_prev = tmp
    return dummy.next

def get_kth(curr, k):
    while curr and k > 0:
        curr = curr.next
        k -= 1
    return curr
```

Complexity: Time O(N), Space O(1)

Classic Problems

- Reverse Linked List — LeetCode 206
- Reverse Linked List II — LeetCode 92
- Reverse Nodes in k-Group — LeetCode 25
- Reorder List — LeetCode 143

Pitfalls

- Losing the `next` pointer before redirecting it.
- Off-by-one on position indices (1-indexed vs 0-indexed).
- Not attaching the dummy node's tail to the remaining list after reversal.

3.7 BFS (Tree + Grid + Graph)

Trigger / When to Use

> **When you see**: shortest path in unweighted graph, minimum steps, level-order traversal, nearest X in grid, connected components (layer by layer).

Core Idea Explore nodes level by level using a queue. Guarantees shortest path in unweighted graphs. Track visited set to avoid revisiting.

Template — Tree Level-Order

```python
from collections import deque

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

Template — Grid BFS

```python
def bfs_grid(grid, start_r, start_c):
    rows, cols = len(grid), len(grid[0])
    queue = deque([(start_r, start_c, 0)])   # (row, col, distance)
    visited = {(start_r, start_c)}
    directions = [(0,1),(0,-1),(1,0),(-1,0)]
    while queue:
        r, c, dist = queue.popleft()
        if is_target(r, c):
            return dist
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in visited:
                if grid[nr][nc] != WALL:
                    visited.add((nr, nc))
                    queue.append((nr, nc, dist + 1))
    return -1
```

Template — Graph BFS

```python
from collections import defaultdict, deque

def bfs_graph(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

Complexity: Time O(V + E), Space O(V)

Classic Problems

- Binary Tree Level Order Traversal — LeetCode 102
- Rotting Oranges — LeetCode 994
- Number of Islands — LeetCode 200
- Word Ladder — LeetCode 127

Pitfalls

- Not marking visited when enqueuing (can enqueue same node multiple times → TLE or infinite loop).
- Measuring `len(queue)` inside the inner loop (changes as you enqueue children).
- Forgetting to handle disconnected graphs (multiple BFS starts or check all nodes).

3.8 DFS (Tree + Grid + Graph)

Trigger / When to Use

> **When you see**: all paths, detect cycle, count components, flood fill, deep exploration before backtracking.

Core Idea Recursively (or with explicit stack) explore as far as possible down one branch before backtracking. Mark visited to prevent cycles.

Template — Tree DFS

```python
def dfs_tree(node):
    if not node:
        return base_case
    left = dfs_tree(node.left)
    right = dfs_tree(node.right)
    return combine(left, right, node.val)

# Iterative
def dfs_iterative(root):
    stack = [root]
    while stack:
        node = stack.pop()
        process(node)
        if node.right: stack.append(node.right)
        if node.left:  stack.append(node.left)    # left last = processed first
```

Template — Grid DFS

```python
def dfs_grid(grid, r, c, visited):
    rows, cols = len(grid), len(grid[0])
    if r < 0 or r ≥ rows or c < 0 or c ≥ cols:
        return
    if (r, c) in visited or grid[r][c] == 0:
        return
    visited.add((r, c))
    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
        dfs_grid(grid, r + dr, c + dc, visited)
```

Template — Graph DFS / Cycle Detection

```python
def has_cycle(graph, n):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n

    def dfs(u):
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY:    # back edge = cycle
                return True
            if color[v] == WHITE and dfs(v):
                return True
        color[u] = BLACK
        return False

    return any(color[u] == WHITE and dfs(u) for u in range(n))
```

Complexity: Time O(V + E), Space O(V) recursion stack

Classic Problems

- Number of Islands — LeetCode 200
- Max Area of Island — LeetCode 695
- Path Sum II — LeetCode 113
- Course Schedule — LeetCode 207

Pitfalls

- Forgetting to mark visited before recursive call (infinite recursion on cycles).
- Python recursion limit (~1000 default); use `sys.setrecursionlimit` or iterative DFS for large inputs.
- Not restoring visited state when you need to explore all paths (use backtracking instead).

3.9 Backtracking

Trigger / When to Use

> **When you see**: generate all valid configurations, constraint satisfaction (N-Queens, Sudoku), all paths in a graph.

Core Idea Build candidates incrementally. At each step, check if candidate is still valid. If not, prune and backtrack. If complete, record solution.

Template

```python
def backtrack(candidates, result, current, start, *constraints):
    if is_complete(current):        # base case: valid solution
        result.append(list(current))
        return
    for i in range(start, len(candidates)):
        if not is_valid(current, candidates[i]):  # prune
            continue
        current.append(candidates[i])             # choose
        backtrack(candidates, result, current, i + 1, *constraints)
        current.pop()                             # un-choose

# Concrete: Combination Sum (reuse allowed)
def combination_sum(candidates, target):
```

```
    result = []
    def backtrack(start, current, remaining):
        if remaining == 0:
            result.append(list(current))
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            current.append(candidates[i])
            backtrack(i, current, remaining - candidates[i])  # i (not i+1): reuse ok
            current.pop()
    backtrack(0, [], target)
    return result
```

Complexity: Time O(2^N) or O(N!) depending on branching factor; Space O(N) recursion depth

Classic Problems

- Combination Sum — LeetCode 39
- N-Queens — LeetCode 51
- Sudoku Solver — LeetCode 37
- Word Search — LeetCode 79

Pitfalls

- Not making a copy when appending to results (`result.append(current[:])`).
- Passing `i+1` when reuse is allowed (should pass `i`).
- Missing pruning conditions → TLE on large inputs.

3.10 Subsets / Permutations / Combinations

Trigger / When to Use

> **When you see**: all subsets, power set, all permutations, all combinations of size K.

Core Idea For subsets: at each element, decide include or exclude. For permutations: at each position, try all unused elements. Backtracking framework handles both.

Template — Subsets

```python
def subsets(nums):
    result = []
    def backtrack(start, current):
        result.append(list(current))        # every prefix is a subset
        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1, current)
            current.pop()
    backtrack(0, [])
    return result

# Bit manipulation alternative (elegant, no recursion)
def subsets_bit(nums):
    n = len(nums)
    return [[nums[j] for j in range(n) if (i >> j) & 1] for i in range(1 << n)]
```

Template — Permutations

```python
def permutations(nums):
    result = []
    def backtrack(current, remaining):
        if not remaining:
            result.append(list(current))
            return
        for i in range(len(remaining)):
            current.append(remaining[i])
            backtrack(current, remaining[:i] + remaining[i+1:])
            current.pop()
    backtrack([], nums)
    return result

# In-place swap permutations (more efficient)
def permutations_swap(nums):
    result = []
    def backtrack(start):
        if start == len(nums):
            result.append(nums[:])
            return
        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]
    backtrack(0)
    return result
```

Complexity: Subsets O(2^N * N), Permutations O(N! * N), Space O(N)

Classic Problems

- Subsets — LeetCode 78
- Subsets II (with duplicates) — LeetCode 90
- Permutations — LeetCode 46
- Combinations — LeetCode 77

Pitfalls

- Subsets II: sort first, then skip `nums[i] == nums[i-1]` at the same recursion level.
- Permutations II: use a `used[]` array or sort + skip duplicates.
- Forgetting `list(current)` copy when appending.

3.11 Binary Search (+ On Answer / Monotonic Predicate)

Trigger / When to Use

> **When you see**: sorted array, rotated sorted array, "find minimum/maximum satisfying a condition", search space is monotonic, minimize the maximum / maximize the minimum.

Core Idea Halve the search space each iteration by comparing the midpoint. "Binary search on answer": define a predicate `feasible(x)` that is monotonically True/False over the answer space, then binary search on that space.

Template — Classic

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left ⩽ right:
        mid = left + (right - left) // 2    # avoids overflow
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Find leftmost occurrence
def lower_bound(arr, target):
    left, right = 0, len(arr)
    while left < right:
        mid = (left + right) // 2
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left
```

Template — On Answer

```python
def binary_search_on_answer(lo, hi, feasible):
    """Find the smallest x in [lo, hi] where feasible(x) is True."""
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid          # mid could be the answer, keep it
        else:
            lo = mid + 1
    return lo

# Example: Koko Eating Bananas — LeetCode 875
def min_eating_speed(piles, h):
    def can_finish(speed):
        return sum((p + speed - 1) // speed for p in piles) ⩽ h
    return binary_search_on_answer(1, max(piles), can_finish)
```

Complexity: Time O(log N) classic, O(log(answer_range) * verification_cost) on answer

Classic Problems

- Binary Search — LeetCode 704
- Search in Rotated Sorted Array — LeetCode 33
- Koko Eating Bananas — LeetCode 875
- Find Minimum in Rotated Sorted Array — LeetCode 153

Pitfalls

- `mid = (left + right) // 2` can overflow in other languages; use `left + (right - left) // 2`.
- Off-by-one: `left ⩽ right` vs `left < right` — use ⩽ for classic, < for leftmost/rightmost variants.
- Binary search on answer: define `feasible` correctly (monotonic), and set `hi` to a valid upper bound.

3.12 Top-K / Heap

Trigger / When to Use

> **When you see**: top K largest, K smallest, K closest points, Kth largest/smallest element, K most frequent.

Core Idea Use a min-heap of size K for "top K largest" (smaller elements get evicted). Use a max-heap for "bottom K smallest". `heapq` in Python is a min-heap; negate values for max-heap.

Template

```python
import heapq

# Top K largest elements — O(N log K)
def top_k_largest(nums, k):
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)      # evict smallest
    return list(min_heap)

# Kth largest — O(N log K)
def kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]  # or use the heap above

# K closest points to origin — O(N log K)
def k_closest(points, k):
    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)

# K most frequent — O(N log K)
from collections import Counter
def top_k_frequent(nums, k):
    count = Counter(nums)
    return heapq.nlargest(k, count.keys(), key=count.get)
```

Complexity: Time O(N log K), Space O(K)

Classic Problems

- Kth Largest Element in an Array — LeetCode 215
- K Closest Points to Origin — LeetCode 973
- Top K Frequent Elements — LeetCode 347
- Find K Pairs with Smallest Sums — LeetCode 373

Pitfalls

- Python's `heapq` is min-heap only; negate for max-heap (`heapq.heappush(h, -val)`).
- `heapq.nlargest(k, ...)` is O(N log K); fine for single calls but build the heap manually for streams.
- For Kth largest, min-heap of size K is more memory-efficient than sorting.

3.13 K-way Merge

Trigger / When to Use

> **When you see**: merge K sorted lists/arrays, find smallest range covering K lists, smallest in range.

Core Idea Use a min-heap with one element from each list. Pop the minimum, push the next element from that list. Repeat.

Template

```python
import heapq

def merge_k_sorted_lists(lists):
    """Merge K sorted linked lists into one sorted list."""
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode(0)
    curr = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next

def merge_k_sorted_arrays(arrays):
    heap = [(arrays[i][0], i, 0) for i in range(len(arrays)) if arrays[i]]
    heapq.heapify(heap)
    result = []
    while heap:
        val, arr_idx, elem_idx = heapq.heappop(heap)
        result.append(val)
        if elem_idx + 1 < len(arrays[arr_idx]):
            heapq.heappush(heap, (arrays[arr_idx][elem_idx+1], arr_idx, elem_idx+1))
    return result
```

Complexity: Time O(N log K) where N = total elements, Space O(K)

Classic Problems

- Merge K Sorted Lists — LeetCode 23
- Find K Pairs with Smallest Sums — LeetCode 373
- Smallest Range Covering Elements from K Lists — LeetCode 632
- Kth Smallest Element in a Sorted Matrix — LeetCode 378

Pitfalls

- Heap tuple must have unique tiebreakers if values can be equal (add index `i` to tuple).
- Don't push `None` into the heap when a list is exhausted.

3.14 Two Heaps (Median of Stream)

Trigger / When to Use

> **When you see**: median of a data stream, balance two halves, sliding window median.

Core Idea Maintain a max-heap for the lower half and a min-heap for the upper half. Always balance sizes so they differ by at most 1. Median is the top of the larger heap (or average of both tops).

Template

```python
import heapq

class MedianFinder:
    def __init__(self):
        self.small = []   # max-heap (lower half): store negated values
        self.large = []   # min-heap (upper half)

    def add_num(self, num):
        heapq.heappush(self.small, -num)          # push to lower half
        # Ensure max(small) ≤ min(large)
        if self.small and self.large and (-self.small[0] > self.large[0]):
            heapq.heappush(self.large, -heapq.heappop(self.small))
        # Balance sizes
        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        elif len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2.0
```

Complexity: Time O(log N) per insertion, O(1) median query; Space O(N)

Classic Problems

- Find Median from Data Stream — LeetCode 295
- Sliding Window Median — LeetCode 480
- IPO (maximize capital) — LeetCode 502

Pitfalls

- Forgetting to negate when pushing to `small` (Python heap is min-heap).
- Rebalancing order matters: first move invalid element, then balance sizes.
- Sliding window median needs lazy deletion (mark removed elements, skip when popped).

3.15 Monotonic Stack

Trigger / When to Use

> **When you see**: next greater/smaller element, previous greater/smaller, stock span, daily temperatures, largest rectangle in histogram, trapping rain water.

Core Idea Maintain a stack that is strictly increasing or decreasing. When a new element violates the order, pop elements (they've found their "next greater/smaller" answer).

Template — Next Greater Element

```python
def next_greater_element(nums):
    n = len(nums)
    result = [-1] * n
    stack = []   # stores indices; values are decreasing (monotonic decreasing)
    for i in range(n):
        while stack and nums[stack[-1]] < nums[i]:
            idx = stack.pop()
            result[idx] = nums[i]          # nums[i] is the next greater for idx
```

```
        stack.append(i)
    return result

def daily_temperatures(temps):
    result = [0] * len(temps)
    stack = []
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            idx = stack.pop()
            result[idx] = i - idx
        stack.append(i)
    return result

def largest_rectangle_histogram(heights):
    stack = []
    max_area = 0
    heights.append(0)           # sentinel to flush stack
    for i, h in enumerate(heights):
        start = i
        while stack and stack[-1][1] > h:
            idx, height = stack.pop()
            max_area = max(max_area, height * (i - idx))
            start = idx
        stack.append((start, h))
    return max_area
```

Complexity: Time O(N), Space O(N)

Classic Problems

- Daily Temperatures — LeetCode 739
- Next Greater Element I — LeetCode 496
- Largest Rectangle in Histogram — LeetCode 84
- Trapping Rain Water — LeetCode 42

Pitfalls

- Deciding increasing vs decreasing: next greater → decreasing stack; next smaller → increasing stack.
- Circular arrays: iterate `2*N` with `i % N` indexing.
- Histogram: store (start_index, height) not just height, to track span.

3.16 Prefix Sum

Trigger / When to Use

> **When you see**: subarray sum equals K, range sum queries, count subarrays with given sum/property.

Core Idea `prefix[i]` = sum of first `i` elements. Range sum `[l, r]` = `prefix[r+1] - prefix[l]`. For count of subarrays summing to K, use a hashmap of prefix sums seen so far.

Template

Python

```
# Range sum query — O(1) per query after O(N) preprocessing
def build_prefix(nums):
    prefix = [0] * (len(nums) + 1)
```

```
    for i, num in enumerate(nums):
        prefix[i+1] = prefix[i] + num
    return prefix

def range_sum(prefix, l, r):           # inclusive [l, r]
    return prefix[r+1] - prefix[l]

# Count subarrays with sum == k — O(N)
from collections import defaultdict
def subarray_sum_equals_k(nums, k):
    count = 0
    prefix = 0
    seen = defaultdict(int)
    seen[0] = 1                        # empty prefix
    for num in nums:
        prefix += num
        count += seen[prefix - k]      # how many prefixes end at (prefix - k)
        seen[prefix] += 1
    return count
```

Complexity: Build O(N), Query O(1); Subarray count O(N), Space O(N)

Classic Problems

- Subarray Sum Equals K — LeetCode 560
- Range Sum Query - Immutable — LeetCode 303
- Count of Range Sum — LeetCode 327
- Product of Array Except Self — LeetCode 238

Pitfalls

- Initialize `seen[0] = 1` (the empty prefix with sum 0 exists once).
- For 2D prefix sums, use inclusion-exclusion: `pre[r2][c2] - pre[r1-1][c2] - pre[r2][c1-1] + pre[r1-1][c1-1]`.
- Negative numbers are fine; this works regardless.

3.17 Union-Find (DSU)

Trigger / When to Use

> **When you see**: connected components, dynamic connectivity, cycle detection in undirected graph, redundant connection, accounts merge.

Core Idea Each element points to a parent. `find(x)` returns root of x's component (with path compression). `union(x, y)` merges components (with rank/size to keep tree flat).

Template

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])   # path compression
```

```
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False                              # already connected
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px                          # union by rank
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        self.components -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

Complexity: Near O(1) amortized per operation (inverse Ackermann α(N)); Space O(N)

Classic Problems

- Number of Connected Components in Undirected Graph — LeetCode 323
- Redundant Connection — LeetCode 684
- Accounts Merge — LeetCode 721
- Making a Large Island — LeetCode 827

Pitfalls

- Not using both path compression AND union by rank (without both, worst case degrades to O(N)).
- Forgetting to check if `find(x) == find(y)` before union (cycle detection).
- Off-by-one with node numbering (0-indexed vs 1-indexed).

3.18 Topological Sort

Trigger / When to Use

> **When you see**: course prerequisites, task scheduling, build dependency order, DAG ordering, alien dictionary.

Core Idea Kahn's algorithm (BFS): compute in-degrees, start with 0 in-degree nodes, process each and reduce neighbors' in-degrees. **DFS-based**: finish time ordering — DFS, push to stack on exit, reverse the stack.

Template — Kahn's (BFS)

Python

```
from collections import deque, defaultdict

def topo_sort_kahn(n, prerequisites):
    graph = defaultdict(list)
    in_degree = [0] * n
    for a, b in prerequisites:          # b → a (b must come before a)
        graph[b].append(a)
        in_degree[a] += 1
    queue = deque([i for i in range(n) if in_degree[i] == 0])
    order = []
```

```
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == n else []  # empty = cycle exists
```

Template — DFS-based

```python
def topo_sort_dfs(n, graph):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n
    result = []
    has_cycle = [False]

    def dfs(u):
        if has_cycle[0]: return
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY:
                has_cycle[0] = True; return
            if color[v] == WHITE:
                dfs(v)
        color[u] = BLACK
        result.append(u)

    for u in range(n):
        if color[u] == WHITE:
            dfs(u)
    return [] if has_cycle[0] else result[::-1]
```

Complexity: Time O(V + E), Space O(V + E)

Classic Problems

- Course Schedule — LeetCode 207
- Course Schedule II — LeetCode 210
- Alien Dictionary — LeetCode 269
- Sequence Reconstruction — LeetCode 444

Pitfalls

- Cycle = no valid ordering: `len(order) == n` check in Kahn's detects it.
- Building graph in wrong direction (which node blocks which).
- Alien Dictionary: adjacent characters in each word define edges; must check invalid input (longer word before shorter prefix → return "").

3.19 Trie

Trigger / When to Use

> **When you see**: word search, prefix matching, autocomplete, count words with prefix, replace words with shortest root.

Core Idea A tree where each node represents a character. Each root-to-leaf path spells a

word. Supports O(L) insert/search (L = word length) regardless of dictionary size.

Template

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        node = self._find_node(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        return self._find_node(prefix) is not None

    def _find_node(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

Complexity: Time O(L) per operation (L = word length); Space O(total characters * alphabet size)

Classic Problems

- Implement Trie (Prefix Tree) — LeetCode 208
- Word Search II — LeetCode 212
- Replace Words — LeetCode 648
- Design Search Autocomplete System — LeetCode 642

Pitfalls

- Using a dict vs array for children: dict is flexible; `[None]*26` is faster for lowercase English only.
- `search` vs `starts_with`: `is_end` flag differentiates them.
- Word Search II: prune the Trie as words are found to avoid revisiting.

3.20 Dynamic Programming

Trigger / When to Use

> **When you see**: count ways, min/max cost/steps, optimal value, overlapping subproblems (same sub-calculation repeated), optimal substructure (optimal solution uses optimal

sub-solutions).

Core Idea Break problem into subproblems; store results (memoization/tabulation). Key: define `dp[i]` or `dp[i][j]` precisely, write the recurrence, identify base cases.

DP --- 0/1 Knapsack

Trigger: "pick or skip" each item, capacity constraint, maximize/count value.

```python
def knapsack_01(weights, values, capacity):
    n = len(weights)
    # dp[i][w] = max value using first i items with capacity w
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            # skip item i
            dp[i][w] = dp[i-1][w]
            # take item i (if it fits)
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1])
    return dp[n][capacity]

# Space-optimized to 1D (iterate w in reverse!)
def knapsack_01_1d(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for i in range(len(weights)):
        for w in range(capacity, weights[i] - 1, -1):   # REVERSE to avoid reuse
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[capacity]
```

Classic: 0/1 Knapsack, Subset Sum (LeetCode 416), Partition Equal Subset Sum (LeetCode 416), Target Sum (LeetCode 494).

DP --- Unbounded Knapsack

Trigger: "unlimited copies" of each item, coin change (any number of coins).

```python
def unbounded_knapsack(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for w in range(1, capacity + 1):
        for i in range(len(weights)):
            if weights[i] <= w:
                dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[capacity]

def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Classic: Coin Change (LeetCode 322), Coin Change II (LeetCode 518), Rod Cutting.
Key difference from 0/1: iterate `w` forward (allows reuse of same item).

DP --- LCS (Longest Common Subsequence)

Trigger: two strings/sequences, longest common subsequence, edit distance, shortest common supersequence.

```python
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

def edit_distance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]
```

Classic: LCS (LeetCode 1143), Edit Distance (LeetCode 72), Shortest Common Supersequence (LeetCode 1092).

DP --- LIS (Longest Increasing Subsequence)

Trigger: longest strictly increasing subsequence, patience sorting, number of LIS.

```python
def lis_dp(nums):
    """O(N^2) DP"""
    n = len(nums)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

def lis_binary_search(nums):
    """O(N log N) using patience sorting"""
    from bisect import bisect_left
    tails = []
    for num in nums:
        pos = bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

Classic: Longest Increasing Subsequence (LeetCode 300), Russian Doll Envelopes (Leet-

Code 354), Number of LIS (LeetCode 673).

DP --- Matrix / Grid DP

Trigger: grid, min path sum, unique paths, count paths, max square of 1s.

```python
def min_path_sum(grid):
    m, n = len(grid), len(grid[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = grid[0][0]
    for i in range(1, m): dp[i][0] = dp[i-1][0] + grid[i][0]
    for j in range(1, n): dp[0][j] = dp[0][j-1] + grid[0][j]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
    return dp[m-1][n-1]

def maximal_square(matrix):
    if not matrix: return 0
    m, n = len(matrix), len(matrix[0])
    dp = [[0]*n for _ in range(m)]
    max_side = 0
    for i in range(m):
        for j in range(n):
            if matrix[i][j] == '1':
                if i == 0 or j == 0:
                    dp[i][j] = 1
                else:
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
                max_side = max(max_side, dp[i][j])
    return max_side * max_side
```

Classic: Unique Paths (LeetCode 62), Minimum Path Sum (LeetCode 64), Maximal Square (LeetCode 221), Dungeon Game (LeetCode 174).

DP --- Interval DP

Trigger: merge stones, burst balloons, matrix chain multiplication, palindrome partitioning.

```python
def burst_balloons(nums):
    """LeetCode 312 — classic interval DP"""
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0]*n for _ in range(n)]
    for length in range(2, n):              # interval length
        for left in range(n - length):
            right = left + length
            for k in range(left+1, right):   # k = last balloon to burst in (left, right)
                dp[left][right] = max(
                    dp[left][right],
                    dp[left][k] + nums[left]*nums[k]*nums[right] + dp[k][right]
                )
    return dp[0][n-1]
```

Classic: Burst Balloons (LeetCode 312), Minimum Cost to Merge Stones (LeetCode 1000), Strange Printer (LeetCode 664).

DP --- Subsequences / Strings

Trigger: count distinct subsequences, palindromic subsequences, word break.

```python
def num_distinct_subsequences(s, t):
    """Count distinct subsequences of s equal to t — LeetCode 115"""
    m, n = len(s), len(t)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = 1     # empty t matched
    for i in range(1, m+1):
        for j in range(1, n+1):
            dp[i][j] = dp[i-1][j]         # don't use s[i-1]
            if s[i-1] == t[j-1]:
                dp[i][j] += dp[i-1][j-1]  # use s[i-1]
    return dp[m][n]

def word_break(s, word_dict):
    """LeetCode 139"""
    word_set = set(word_dict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n+1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True; break
    return dp[n]
```

Classic: Distinct Subsequences (LeetCode 115), Word Break (LeetCode 139), Palindromic Substrings (LeetCode 647).

3.21 Greedy

Trigger / When to Use

> **When you see**: ”minimum number of X”, ”maximize Y”, local decisions that don't affect future options, exchange argument, interval scheduling.

Core Idea Make the locally optimal choice at each step. Prove (or intuit) that no better global solution exists. Common proof technique: ”exchange argument” — show swapping any element for the greedy choice can only improve or maintain the solution.

Template — Interval Scheduling Maximization

```python
def max_non_overlapping_intervals(intervals):
    """Maximum number of non-overlapping intervals (greedy: earliest end first)"""
    intervals.sort(key=lambda x: x[1])   # sort by END time
    count = 0
    last_end = float('-inf')
    for start, end in intervals:
        if start ≥ last_end:
            count += 1
            last_end = end
    return count

def jump_game(nums):
    """Can reach last index? — LeetCode 55"""
```

```
    max_reach = 0
    for i, num in enumerate(nums):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + num)
    return True

def jump_game_ii(nums):
    """Minimum jumps to reach last index — LeetCode 45"""
    jumps = cur_end = cur_far = 0
    for i in range(len(nums) - 1):
        cur_far = max(cur_far, i + nums[i])
        if i == cur_end:
            jumps += 1
            cur_end = cur_far
    return jumps
```

Complexity: Usually O(N log N) (sort) + O(N); Space O(1)

Classic Problems

- Jump Game — LeetCode 55
- Non-overlapping Intervals — LeetCode 435
- Task Scheduler — LeetCode 621
- Gas Station — LeetCode 134

Pitfalls

- Not all greedy approaches are correct; verify with a counterexample.
- Activity selection: sort by END (not start, not duration).
- When greedy fails, fall back to DP.

3.22 Bit Manipulation

Trigger / When to Use

> **When you see**: find unique element in array where all others appear twice/three times, count set bits, power of 2, XOR tricks, subset enumeration.

Core Idea Exploit binary properties: XOR cancels duplicates, AND/OR extract/set bits, shifting multiplies/divides by powers of 2.

Key Tricks

Python

```
# XOR: a ^ a = 0, a ^ 0 = a
def single_number(nums):
    """Find the element appearing once — LeetCode 136"""
    result = 0
    for num in nums:
        result ^= num
    return result

# Count set bits (Brian Kernighan)
def count_bits(n):
    count = 0
    while n:
        n &= (n - 1)       # removes lowest set bit
```

```
        count += 1
    return count

# Check power of 2
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

# Get / Set / Clear bit
def get_bit(n, i):   return (n >> i) & 1
def set_bit(n, i):   return n | (1 << i)
def clear_bit(n, i): return n & ~(1 << i)

# Enumerate all subsets of a bitmask
def enumerate_subsets(mask):
    sub = mask
    while sub:
        process(sub)
        sub = (sub - 1) & mask    # next smaller submask

# XOR to find two unique numbers (all others appear twice)
def single_number_iii(nums):
    """LeetCode 260"""
    xor = 0
    for n in nums: xor ^= n           # xor of the two unique numbers
    diff_bit = xor & (-xor)           # rightmost set bit
    a = b = 0
    for n in nums:
        if n & diff_bit: a ^= n
        else:            b ^= n
    return [a, b]
```

Complexity: O(N) time, O(1) space for most tricks

Classic Problems

- Single Number — LeetCode 136
- Number of 1 Bits — LeetCode 191
- Counting Bits — LeetCode 338
- Reverse Bits — LeetCode 190
- Single Number III — LeetCode 260

Pitfalls

- Python integers have arbitrary precision; no overflow, but be careful with `~n` (gives `-n - 1`).
- Use `n & (-n)` to isolate the lowest set bit.
- Bitmask DP: remember `1 << n` can be large; check constraints.

4 Complexity Quick Reference

4.1 Big-O of Common Data Structure Operations

Operation	Array	Linked List	Hash Map	BST (avg)	Heap	Sorted Array
Access by index	O(1)	O(N)	N/A	N/A	N/A	O(1)
Search	O(N)	O(N)	O(1)	O(log N)	O(N)	O(log N)
Insert (end)	O(1)*	O(1)	O(1)	O(log N)	O(log N)	O(N)

Operation	Array	Linked List	Hash Map	BST (avg)	Heap	Sorted Array
Insert (middle)	O(N)	O(1)**	O(1)	O(log N)	O(log N)	O(N)
Delete	O(N)	O(1)**	O(1)	O(log N)	O(log N)	O(N)
Min/Max	O(N)	O(N)	O(N)	O(log N)	O(1)	O(1)

*Amortized for dynamic array. **Given a pointer to the node.

4.2 Common Algorithm Complexities

Algorithm	Time	Space
Binary Search	O(log N)	O(1)
Quick Sort	O(N log N) avg, $O(N^2)$ worst	O(log N)
Merge Sort	O(N log N)	O(N)
Heap Sort	O(N log N)	O(1)
BFS / DFS	O(V + E)	O(V)
Dijkstra (heap)	O((V+E) log V)	O(V)
Topological Sort	O(V + E)	O(V)
Union-Find	$O(\alpha(N))$ per op	O(N)
Trie insert/search	O(L)	O(L)

4.3 Acceptable Complexity vs Input Size N

N (input size)	Max acceptable complexity	Notes
$N \leq 10$	O(N!)	Permutations, backtracking with full exploration
$N \leq 20$	O(2^N)	Bitmask DP, subset enumeration
$N \leq 100$	$O(N^3)$	Matrix chain, interval DP, Floyd-Warshall
$N \leq 500$	$O(N^2)$	Some DP, all-pairs work
$N \leq 1{,}000$	$O(N^2)$	Bubble/insertion sort if needed, $O(N^2)$ DP
$N \leq 10{,}000$	$O(N^2 \log N)$	Rarely; $O(N^2)$ should work
$N \leq 100{,}000$	O(N log N)	Sort, heap operations, segment tree
$N \leq 1{,}000{,}000$	O(N log N)	Must be near-linear
$N \leq 10{,}000{,}000$	O(N)	Linear only — prefix sum, two pointers, sliding window
N > 10^8	O(log N) or O(1)	Binary search, math tricks

> **Rule**: Assume ~10^8 simple operations per second for Python (10^9 for C++). Multiply complexity by constant factors; Python is ~5-10x slower than C++.

5 Interview Communication Tips

5.1 Before You Code

1. **Repeat the problem** in your own words. "So we're given... and we need to return..."
2. **Clarify edge cases** out loud: empty input? negative numbers? duplicates allowed? integer overflow?
3. **State constraints** you noticed: "N can be up to 10^5, so we need O(N log N) or better."
4. **Start with brute force** — even if obvious: "The naive approach is $O(N^2)$ by checking all pairs. We can do better..."
5. **Think aloud** while optimizing: "I notice the array is sorted, so maybe we can binary search... or use two pointers..."

5.2 While Coding

- **Name variables clearly**: `left`, `right`, `window_sum`, not `i`, `j`, `temp`.
- **Comment key invariants**: `# left: next position to fill`, `# window is always valid here`.
- **Write helper functions** for repetitive logic rather than duplicating code.
- **Don't go silent** for more than 30 seconds; narrate your thought process.

5.3 After Coding

1. **Trace through your example** step by step on the code you wrote.
2. **Check edge cases**: empty array, single element, all same, already sorted, negative numbers.
3. **State complexity** proactively: "Time is O(N log N) for the sort plus O(N) for the scan, so O(N log N). Space is O(1) if we ignore the output."
4. **Ask if they want you to optimize** before jumping to a different approach.

5.4 Recovery Tactics

- Stuck? Say "Let me think about what information I'm not using yet."
- Hint? "That's helpful — so if we track X, we could avoid recomputing Y..."
- Bug? "Let me re-trace this test case to find where it diverges."

6 Typical Mistakes Candidates Make

Mistake	Fix
Jumping to code without clarifying	Always spend 2-3 min clarifying before touching code
Skipping brute force	State it first; it demonstrates structured thinking and sets a baseline
Not handling edge cases	Build a checklist: empty, single element, all same, negatives, overflow
Vague variable names (`i`, `j`, `tmp`)	Use descriptive names even if it takes 5 more seconds
Not stating complexity	Interviewers almost always ask; volunteer it proactively
Silently debugging	Narrate: "I think the bug is here because..."
Confusing inclusive/exclusive bounds	Write `# inclusive` or `# exclusive` in comments
Off-by-one in binary search	Stick to one template and test `[1]`, `[1,2]`, `[1,2,3]`
Forgetting to copy list before appending	`result.append(current[:])` not `result.append(current)`
Not tracking the `visited` set	Add it before BFS/DFS; mark visited when enqueuing
Wrong loop order in DP	0/1 knapsack: reverse; unbounded: forward

Mistake	Fix
Greedy without justification	Briefly argue why greedy works or think about exchange argument
Python recursion limit	Add `sys.setrecursionlimit(10**6)` for deep DFS
Mutating input	Ask "can I modify the input?" or work on a copy

7 Revision Cheat Sheet

7.1 The 20 Must-Know Patterns

#	Pattern	1-line Memory Hook
1	**Two Pointers**	Sorted + pair/triplet → shrink from both ends
2	**Sliding Window**	Contiguous subarray condition → shrink/grow window
3	**Fast & Slow Pointers**	Cycle / middle → tortoise and hare
4	**Merge Intervals**	Overlapping intervals → sort by start, merge greedily
5	**Cyclic Sort**	Array with [1..N] → each number belongs at index n-1
6	**LL Reversal**	Reverse linked list → prev/curr/next dance
7	**BFS**	Shortest path / level order → queue + visited
8	**DFS / Backtracking**	All paths / all configs → recurse + undo
9	**Binary Search**	Sorted / monotonic predicate → halve search space
10	**Top-K Heap**	K largest/smallest → min-heap of size K
11	**K-way Merge**	K sorted lists → heap with one from each
12	**Two Heaps**	Median of stream → max-heap left, min-heap right
13	**Monotonic Stack**	Next greater/smaller → pop when violated
14	**Prefix Sum**	Subarray sum = K → hash map of prefix sums
15	**Union-Find**	Dynamic connectivity → compress + rank
16	**Topo Sort**	DAG ordering / cycle → Kahn's (BFS in-degree)
17	**Trie**	Prefix search → character tree
18	**0/1 Knapsack DP**	Pick or skip + capacity → 2D or reverse-1D DP
19	**Greedy**	Local optimal = global optimal → sort + scan
20	**Bit Manipulation**	XOR / AND / shifts → bitmask tricks

7.2 Decision Tree (30-second pattern picker)

```
Is array sorted or can we sort it?
├── Yes → Two Pointers or Binary Search
└── No
    ├── Subarray/substring condition? → Sliding Window
    ├── Linked list? → Fast & Slow or LL Reversal
    ├── Graph/tree?
    │   ├── Shortest path / min steps? → BFS
    │   ├── All paths / components? → DFS / Backtracking
    │   ├── Dependency order? → Topological Sort
    │   └── Connected components (dynamic)? → Union-Find
    ├── K elements? → Heap (Top-K or K-way Merge)
    ├── Intervals? → Merge Intervals
    ├── Count ways / optimal value? → Dynamic Programming
    ├── Prefix search / autocomplete? → Trie
```

```
    ├── Next greater / histogram? → Monotonic Stack
    ├── Subarray sum queries? → Prefix Sum
    └── XOR / bits / unique? → Bit Manipulation
```

7.3 Quick Complexity Reminder

```
O(1)       → hash map lookup, heap top
O(log N)   → binary search, heap push/pop
O(N)       → single pass, two pointers, sliding window
O(N log N) → sort, N heap operations
O(N²)      → nested loops, some DP
O(2^N)     → subsets, bitmask DP
O(N!)      → permutations
```

Last updated: 2026-06-14 | Covers ~25 patterns with 100+ LeetCode problems

8 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

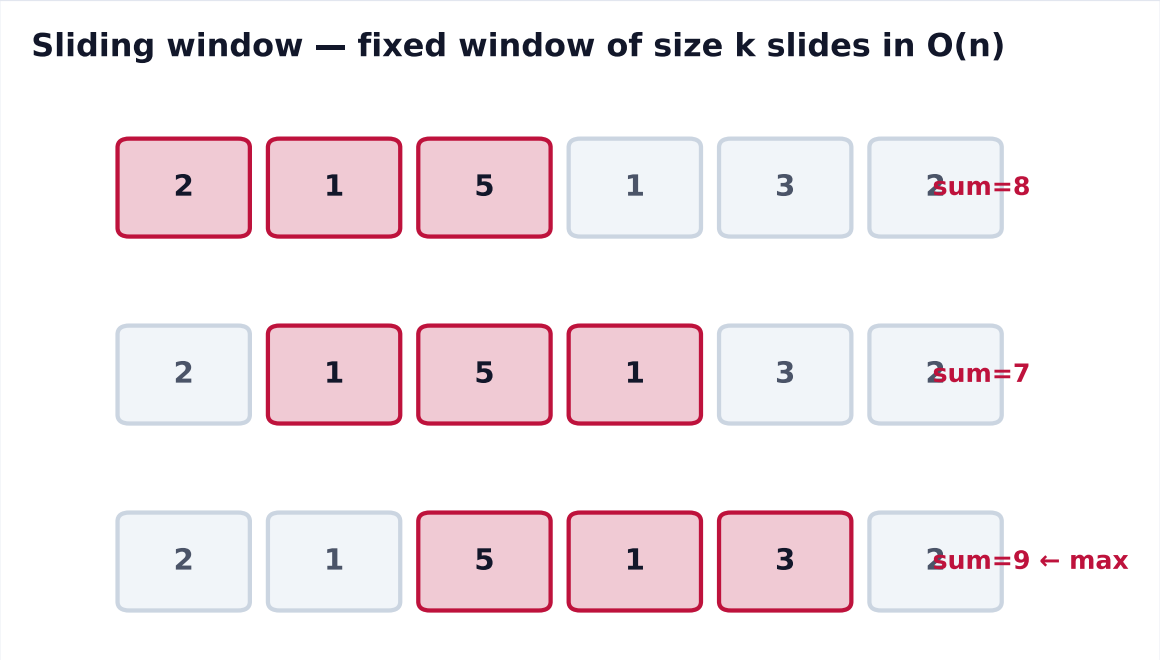

Figure 1. Instead of recomputing each window from scratch (O(n·k)), slide the window: add the entering element and drop the leaving one, achieving O(n).

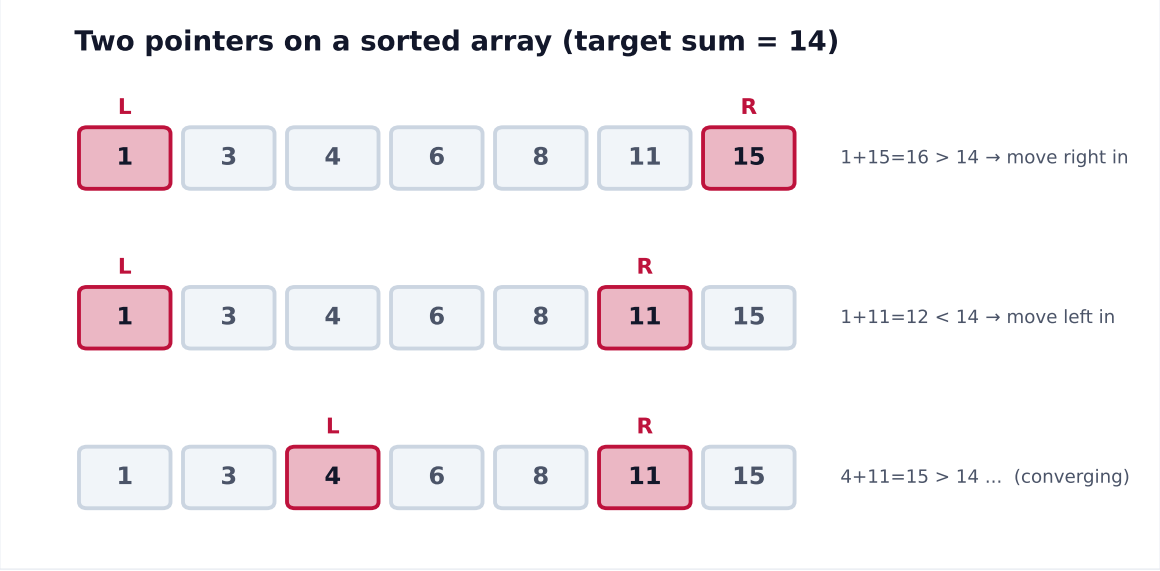

Figure 2. On sorted input, two pointers from both ends turn an $O(n^2)$ pair-search into O(n): move the pointer that brings the sum toward the target.